VERIQTA

NGINX for DevOps Engineers

30 Real Configurations and Fixes



- REVERSE PROXY
- LOAD BALANCING
- SSL / TLS CONFIGURATION
- PERFORMANCE TUNING
- SECURITY HARDENING
- CACHING
- REWRITES & REDIRECTS
- LOGGING & MONITORING
- TROUBLESHOOTING REAL ISSUES
- PRODUCTION BEST PRACTICES

@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

VERIQTA

Nginx Is Running, but Production Traffic Is Failing

Configuration 01 — Diagnose a Healthy Nginx Process That Returns 502 Errors

Scenario

Nginx is active, but customers receive intermittent **502 Bad Gateway** responses.

Key Points

1. Distinguish an active Nginx process from a healthy upstream application.
2. Investigate refused connections, premature upstream closure, and invalid upstream responses.
3. Correlate Nginx error logs with upstream service health.
4. Use **ss -lntp**, **curl**, and **journalctl** to isolate the failure.
5. Fix incorrect upstream addresses, unhealthy services, or connection handling based on evidence.
6. Verify successful responses and reduced upstream errors.

Architecture Overview



1. Verify Nginx Process and Listening Ports

```
# Check Nginx service status
sudo systemctl status nginx

# Check if Nginx is listening on the expected ports
sudo ss -lntp | grep nginx
```

Look for:

- Service is active (running)
- Listening on 80/443 or your configured ports
- No obvious startup errors

2. Test Nginx Response and Upstream Application

```
# Test through Nginx (public or local)
curl -I http://your-domain.com

# Test the upstream application directly
curl -I http://upstream-ip:8080

# Test with verbose output
curl -v http://your-domain.com
```

Expected Results:

- If Nginx returns 502 but upstream fails or refuses connection, the issue is upstream.
- If upstream is healthy directly but fails via Nginx, check Nginx configuration and networking.

3. Investigate Nginx Error Logs

```
# View recent error logs
sudo tail -n 100 /var/log/nginx/error.log

# Follow logs in real time
sudo tail -f /var/log/nginx/error.log
```

Common 502 Error Messages:

- connect() failed (111: Connection refused)
- upstream prematurely closed connection
- invalid header from upstream
- no live upstreams while connecting
- upstream timed out (110: Connection timed out)

4. Check Application and System Logs

```
# Check application logs (example)
sudo journalctl -u app-service -n 100

# Check application status
sudo systemctl status app-service

# Check upstream port on the server
sudo ss -lntp | grep :8080
```

5. Fix the Issue

- Correct upstream IP address or hostname
- Start or restart unhealthy application services
- Fix network connectivity or firewall rules
- Update Nginx upstream configuration
- Adjust timeouts or connection settings if needed

Example Upstream Configuration

```
upstream app_backend {
    server 10.0.1.10:8080;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080;
    keepalive 32;
}

server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://app_backend;
        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
        proxy_send_timeout 30s;
    }
}
```

6. Verify the Fix

- Test the application through Nginx again
- Confirm 200 OK responses
- Monitor error logs for reduced 502 errors
- Check upstream service health and metrics
- Validate from multiple locations (internal and external)

Senior Insight

A healthy reverse proxy does not prove that the application behind it is healthy.

VERIQTA

The Reverse Proxy That Exposed the Wrong Application

Configuration 02 — Build Deterministic Virtual Host Routing

Scenario

A request intended for one application reaches another application hosted on the same Nginx server.

Key Points

1. Understand server_name, listen, and default-server selection.
2. Examine unmatched hostnames and unintended fallback behavior.
3. Configure explicit virtual hosts and a safe default server.
4. Test routing with curl --resolve.
5. Prevent accidental exposure of internal applications.

Nginx Virtual Host Configuration (Correct)

```
# Application 1: app1.example.com
server {
    listen 80;
    server_name app1.example.com;

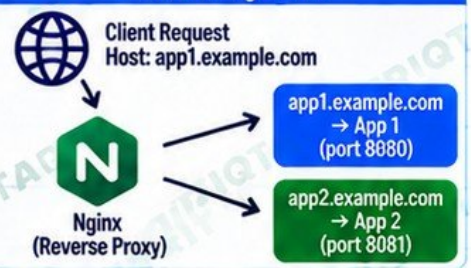
    location / {
        proxy_pass http://app1_backend;
    }
}

# Application 2: app2.example.com
server {
    listen 80;
    server_name app2.example.com;

    location / {
        proxy_pass http://app2_backend;
    }
}

# Safe default server for unmatched hostnames
server {
    listen 80 default_server;
    server_name _;
    return 444;  # or return 404;
}
```

How Routing Works



Common Issues

- No matching server_name
- Missing default server
- Requests falling back to the wrong app
- Internal app exposed
- Wildcard server_name misconfiguration

Test Routing with curl --resolve

```
# Test app1
curl --resolve app1.example.com:80:SERVER_IP \
   -H "Host: app1.example.com" http://app1.example.com
# Test app2
curl --resolve app2.example.com:80:SERVER_IP \
   -H "Host: app2.example.com" http://app2.example.com
```

Configuration 03 — Fix Dangerous Location Matching

Scenario

An authentication endpoint is routed through a less restrictive Nginx location.

Key Points

1. Understand exact, prefix, ^~, and regex location selection.
2. Examine how regex locations can override ordinary prefix matches.
3. Build deterministic routing for /api/, /admin/, and static assets.
4. Verify behavior using controlled request paths.

Location Matching Examples

```
server {
    listen 80;
    server_name example.com;

    # Exact match (highest priority)
    location = /admin {
        return 403;
    }

    # Prefix match, no regex override
    location ^~ /api/ {
        proxy_pass http://api_backend;
    }
    # Regular prefix match
    location / {
        proxy_pass http://app_backend;
    }
    # Regex location (evaluated after prefix unless ^~)
    location ~* \.(jpg|jpeg|png|gif|css|js)$ {
        root /var/www/static;
        expires 30d;
    }
}
```

Nginx Location Matching Order

1. Exact match: location = /uri
2. Prefix match with ^~: location ^~ /uri/
3. Regular prefix match: location /uri/
4. Regex match (in order): location ~ /uri/
5. Named locations: location @name

Test Location Matching

```
# Test API path
curl -I http://example.com/api/health

# Test admin path
curl -I http://example.com/admin

# Test static asset
curl -I http://example.com/static/app.js

# Test regex path
curl -I http://example.com/image.jpg
```

Senior Insight — Routing mistakes can become security incidents even when every backend application is correctly configured.

The Proxy That Broke Authentication and Client Identity

Configuration 04 — Preserve the Correct Client IP Behind Multiple Proxies

Scenario

Every request appears to originate from the load balancer.

Key Points

1. Configure set_real_ip_from, real_ip_header, and trusted proxy boundaries.
2. Distinguish genuine client identity from spoofed forwarded headers.
3. Examine AWS ALB and CDN forwarding behavior.
4. Verify access logs contain the correct client address.

Architecture Example



Nginx Configuration Example

```
# Trust only your known proxies
set_real_ip_from 203.0.113.0/24;   # CloudFront
set_real_ip_from 10.0.0.0/16;      # ALB (example)

# Use the header that contains the client IP
real_ip_header X-Forwarded-For;

# Optional: overwrite any existing header
real_ip_recursive on;

# Example server block
server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass http://app_backend;
    }
}
```

Verify Client IP in Access Logs

```
log_format realip '$remote_addr - $http_x_forwarded_for
[$time_local] "$request" $status';
```

Result in Access Log

```
203.0.113.10 - 203.0.113.10, 54.240.1.1
[10/Oct/2026:12:15:30 +0000] "GET / HTTP/1.1" 200
```

Test with curl (simulate client IP)

```
curl -H "X-Forwarded-For: 203.0.113.10" \
     -H "Host: example.com" \
       http://your-nginx-ip/
```

Important Considerations

- Trust only your own proxies (ALB, CDN).
- Do not trust X-Forwarded-For from the internet.
- AWS ALB appends the client IP to X-Forwarded-For.
- CloudFront uses X-Forwarded-For and CloudFront-Viewer-Address.
- Validate logs to confirm the correct client IP.

Configuration 05 — Fix Authentication Redirect Loops Behind TLS Termination

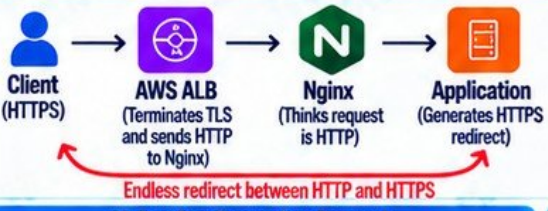
Scenario

Users are repeatedly redirected between HTTP and HTTPS.

Key Points

1. Examine Host, X-Forwarded-Proto, and application URL generation.
2. Configure forwarded headers according to trusted proxy topology.
3. Investigate incorrect redirect schemes and proxy chains.
4. Verify authentication redirects without creating spoofable trust boundaries.

Typical Redirect Loop



Common Causes

- Missing or incorrect X-Forwarded-Proto
- Application not trusting proxy headers
- Misconfigured Host header
- Redirect rule in Nginx forcing HTTPS
- Application generating absolute URLs with the wrong scheme

Nginx Configuration Example

```
server {
    listen 80;
    server_name example.com;

    # Pass the correct forwarded headers
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    location / {
        proxy_pass http://app_backend;
    }
}
```

Test the Fix

```
# Test through the load balancer or directly
curl -I https://example.com/login

# Check for single redirect to HTTPS (no loop)
# Expected: 200 or single 302 to HTTPS

# Verify headers seen by application
curl -H "Host: example.com" \
     - H "X-Forwarded-Proto: https" \
       http://your-nginx-ip/debug
```

Senior Insight — Forwarded headers are claims. Trust them only when they originate from infrastructure you control.

VERIQTA

The Load Balancer That Kept Sending Traffic to a Failing Server

Configuration 06 — Configure Passive Upstream Failure Handling

Scenario

One backend repeatedly fails while other application instances remain healthy.

Key Points

1. Configure upstream, max_fails, and fail_timeout.
2. Understand passive failure detection and its limitations.
3. Examine the behavior of retrying requests across upstream servers.
4. Distinguish Nginx Open Source passive checks from active health-check features available in commercial editions.
5. Verify backend selection through access logs and controlled failure testing.

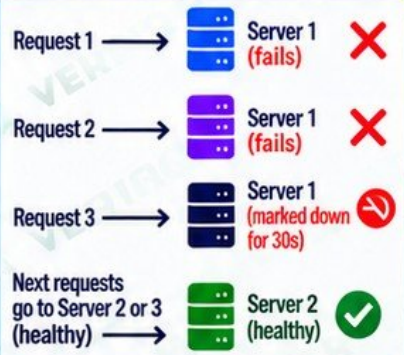
Nginx Upstream Configuration Example

```
upstream app_backend {
  server 10.0.1.10:8080 max_fails=3 fail_timeout=30s;
  server 10.0.1.11:8080 max_fails=3 fail_timeout=30s;
  server 10.0.1.12:8080 max_fails=3 fail_timeout=30s;
}

server {
   listen 80;
   server_name example.com;

   location / {
      proxy_pass http://app_backend;
      proxy_next_upstream error timeout
            http_502 http_503 http_504;
   }
}
```

How Passive Failure Detection Works



Passive vs Active Health Checks

Nginx Open Source (Passive Checks)

- Marks server as failed when requests return errors or time out
- Uses max_fails and fail_timeout
- No independent health check to verify server health
- Relies on real traffic to detect failures

Nginx Plus (Active Checks)

- Performs independent health checks (HTTP, TCP, gRPC)
- Detects failures without waiting for real traffic
- More accurate and faster failover
- Additional configuration and license required

Verify in Access Logs

```
# Log upstream address
log_format upstream_log
'$remote_addr - $host [$time_local] '
'$request $status $upstream_addr '
'$upstream_status $upstream_response_time';

# Example log (after failure)
203.0.113.10 - example.com [10/Oct/2026:12:01:10]
"GET / HTTP/1.1" 200 10.0.1.11:8080
200 0.123
```

Configuration 07 — Prevent Unsafe Retries of Non-Idempotent Requests

Scenario

A payment request is retried after an upstream connection failure.

Key Points

1. Understand proxy_next_upstream and retry eligibility.
2. Examine why repeating a request can duplicate external side effects.
3. Configure bounded retries for appropriate failure conditions.
4. Design application-level idempotency for critical transactions.
5. Verify that unsafe requests are not replayed unexpectedly.

Nginx Configuration with Safe Retry Policy

```
upstream payment_backend {
  server 10.0.2.10:8080 max_fails=3 fail_timeout=30s;
  server 10.0.2.11:8080 max_fails=3 fail_timeout=30s;
}
server {
   listen 80;
   server_name payments.example.com;

   location / {
      proxy_pass http://payment_backend;
      # Retry only for safe conditions
      proxy_next_upstream error timeout http_502 http_503;
      proxy_next_upstream_tries 2;
      proxy_next_upstream_timeout 5s;
   }
}
```

Retry Eligibility

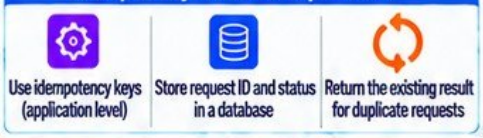
Safe to Retry (Generally)

- Network errors (connection failed) Timeouts
- 502 / 503 / 504 from upstream
- Idempotent requests (GET, HEAD, PUT, DELETE if designed to be idempotent)

Not Safe to Retry

- Non-idempotent requests (POST for payments and create operations)
- Requests that already reached the application
- Operations with external side effects (e.g., charge card, send email, create resource)

Idempotency for Critical Operations



Test and Verify

```
# Simulate upstream failure
sudo iptables -A INPUT -p tcp --dport 8080 -j REJECT
# Send a POST request (should not be retried)
curl -X POST http://payments.example.com/api/charge
# Check access logs to confirm only one request was processed
grep "/api/charge" /var/log/nginx/access.log
```

Senior Insight — Improving availability through retries can introduce data-integrity failures.

VERIQTA

The Backend Is Healthy, but Requests Are Unevenly Distributed

Configuration 08 — Choose the Correct Load Balancing Strategy

Scenario

One application instance experiences sustained overload while others remain underutilized.

Key Points

1. Compare round robin, least_conn, and hash-based distribution.
2. Examine long-running requests and uneven connection durations.
3. Understand why connection count does not always represent CPU or application load.
4. Select balancing strategies based on workload behavior.
5. Measure per-upstream latency and request distribution.

Load Balancing Strategies

Round Robin (Default)

- Distributes requests evenly in sequence.
- Simple and lightweight.
- Suitable for similar instances.
- May not work well for long-running requests.

least_conn

- Sends requests to the server with the fewest active connections.
- Better for long-running requests.
- More dynamic load distribution.
- Connection count does not always equal CPU load.

Hash-based (ip_hash / hash)

- Routes based on a hash (e.g. client IP, header, or key).
- Good for session persistence.
- Ensures the same client reaches the same backend.
- Less balanced if one client is very heavy.

Nginx Configuration Examples

Round Robin (default)

```
upstream app_backend {
    server 10.0.1.10:8080;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080;
}
server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_pass http://app_backend;
    }
}
```

least_conn

```
upstream app_backend {
    least_conn;
    server 10.0.1.10:8080;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080;
}
server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_pass http://app_backend;
    }
}
```

Hash-based (ip_hash)

```
upstream app_backend {
    ip_hash;
    server 10.0.1.10:8080;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080;
}
server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_pass http://app_backend;
    }
}
```

Measure Per-Upstream Latency and Request Distribution

```
log_format upstream_log '$remote_addr - $host [$time_local] '
        '"$request" $status $body_bytes_sent '
        '"$upstream_addr" $upstream_status '
        '"$upstream_response_time" $request_time';
access_log /var/log/nginx/upstream.log upstream_log;
```

Analyze with Command Line

```
# Request distribution
sudo awk '{print $9}' /var/log/nginx/upstream.log |
    cut -d: -f1 | sort | uniq -c

# Compare response times
sudo awk '{print $10}' /var/log/nginx/upstream.log |
    sort -n | tail -10
```

Look For

- Even request distribution
- High latency on one server
- Long-running requests
- Underutilized instances

Configuration 09 — Configure Upstream Connection Reuse Safely

Scenario

Excessive connection establishment increases latency and backend resource usage.

Key Points

1. Configure upstream keepalive connection reuse.
2. Understand HTTP version behavior and connection lifecycle.
3. Examine connection limits and idle connection accumulation.
4. Distinguish client-side keepalive from upstream keepalive.
5. Verify connection reuse and backend capacity.

Nginx Upstream Keepalive Configuration

```
upstream app_backend {
    server 10.0.1.10:8080;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080;
    keepalive 32;  # number of idle keepalive connections
}
server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_pass http://app_backend;
    }
}
```

Important Settings

- keepalive <number> # idle connections per worker
- proxy_http_version 1.1
- proxy_set_header Connection ""
- Tune based on backend capacity

Client-side vs Upstream Keepalive

Client-side Keepalive (client → Nginx)

- Handles connections from clients to Nginx
- Controlled by keepalive_timeout
- Reduces client connection overhead

Upstream Keepalive (Nginx → Backend)

- Reuses connections to backend
- Configured in upstream block
- Reduces backend connection establishment and latency

Verify Connection Reuse

```
# Check established connections to upstream
sudo ss -tnp | grep :8080
# Monitor Nginx upstream connections
sudo tail -f /var/log/nginx/upstream.log
```

Look For

- Fewer new TCP handshakes
- Stable number of idle connections
- Reduced latency
- No excessive idle connections

Common Issues

- Too many idle connections
- Backend hitting connection limits
- Using HTTP/1.0 (no keepalive)
- Not setting proxy_http_version 1.1

Senior Insight — Connection efficiency can improve throughput, but poorly bounded connections can exhaust downstream resources.

VERIQTA

Increasing Nginx Timeouts Made the Outage Worse

Configuration 10 — Configure Timeouts According to Failure Domains

Scenario

Engineers increase timeouts from 30 seconds to 300 seconds, and the system becomes less responsive.

Key Points

1. Distinguish proxy_connect_timeout, proxy_send_timeout, and proxy_read_timeout.
2. These directives govern different connection or I/O intervals, not one universal request deadline.
3. Examine slow backends, stalled responses, and connection accumulation.
4. Coordinate Nginx timeouts with application and client budgets.
5. Verify improvements using upstream response-time metrics.

Nginx Timeout Directives

```
location / {
    proxy_pass http://app_backend;

    proxy_connect_timeout 5s;  # Time to connect
    proxy_send_timeout 30s;  # Time to send request
    proxy_read_timeout 30s;  # Time to read response
}
```

What Each Timeout Controls

proxy_connect_timeout	• Time to establish a TCP connection to the upstream server. • Triggered when the upstream is unreachable or slow to accept connections.
proxy_send_timeout	• Time to send the request to the upstream server (including request body). • Triggered when the upstream is not reading data.
proxy_read_timeout	• Time to wait for a response from the upstream server. • Triggered when the upstream is slow or not sending data.

Why Increasing Timeouts Can Make Things Worse

- More idle connections staying open.
- Increased memory and file descriptor usage.
- More requests waiting for slow backends.
- Connection queues growing.
- Overall system becomes less responsive.

Observe Upstream Response Times

```
# Nginx access log with upstream timing
log_format upstream_timing
    '$remote_addr - $host $request '
    '$status $body_bytes_sent '
    '$request_time $upstream_response_time '
    '$upstream_addr';
```

Example Log Output

```
203.0.113.10 - example.com "GET /api" 200 1024
0.345 0.320 10.0.1.10:8080
```

Verify and Monitor

- Check upstream_response_time and request_time.
- Monitor active connections: sudo ss -s
- Check Nginx stub_status: curl http://localhost/nginx_status
- Use application and upstream metrics (latency, errors).
- Adjust timeouts based on real measurements, not guesses.

Configuration 11 — Protect Nginx Against Slow Clients

Scenario

Slow or malicious clients occupy connections for excessive periods.

Key Points

1. Configure client_header_timeout and client_body_timeout.
2. Examine send_timeout and client-side connection behavior.
3. Balance legitimate slow networks against resource exhaustion.
4. Observe active connections and request completion patterns.

Nginx Configuration for Slow Clients

```
server {
    listen 80;
    server_name example.com;

    client_header_timeout 10s;  # Time to receive headers
    client_body_timeout 20s;  # Time to receive request body
    send_timeout 30s;    # Time to send response to client

    location / {
        proxy_pass http://app_backend;
    }
}
```

What Each Directive Controls

client_header_timeout	• Time to receive the complete request headers from the client. • Protects against clients sending headers very slowly.
client_body_timeout	• Time to receive the full request body (e.g. file uploads). • Prevents slow or stalled uploads from holding connections.
send_timeout	• Time to send the response to the client. • Disconnects slow clients that are not reading the response.

Monitor Active Connections

```
# View connection summary
sudo ss -s

# Check Nginx active connections
curl http://localhost/nginx_status

# View real-time connections
sudo watch -n 1 'ss -tn state
established | wc -l'
```

Detect Slow Clients in Logs

```
log_format client_timing
  '$remote_addr - $host $request '
  '$status $body_bytes_sent '
  '$request_time $upstream_response_time
  '$http_user_agent';

# Find slow requests
sudo awk '$7 > 10 {print}'
/var/log/nginx/access.log
```

Important Considerations

- Use reasonable timeouts to avoid disconnecting legitimate slow networks.
- Set lower limits for public endpoints.
- Combine with rate limiting for additional protection.
- Monitor the rate of client timeouts.
- Tune based on real traffic patterns and client behavior.

Senior Insight — A longer timeout is not additional capacity. It may simply retain failed work for longer.

VERIQTA

Nginx Reached Its Connection Limit Before CPU Became Busy

Configuration 12 — Tune Worker Capacity Without Hiding the Bottleneck

Scenario

Nginx rejects new connections despite moderate CPU utilization.

Key Points

1. Examine worker_processes, worker_connections, and file descriptor limits.
2. Understand how upstream connections also consume connection capacity.
3. Investigate RLIMIT_NOFILE, systemd limits, and kernel socket behavior.
4. Correlate Nginx connection metrics with operating-system evidence.
5. Test capacity under realistic traffic before changing limits.

Nginx Worker Configuration Example

```
worker_processes  auto;  # or set to CPU cores
worker_rlimit_nofile  200000;
events {
   worker_connections 65535;  # per worker
   use epoll;
}
http {
    upstream app_backend {
       server 10.0.1.10:8080;
       server 10.0.1.11:8080;
       keepalive 32;    # upstream keepalive
    }
}
```

Check Current Limits and Usage

```
# Nginx process limits
sudo cat /proc/$(pgrep -n nginx)/limits | grep "open files"
Max open files            1024      200000      files

# Systemd service limits
systemctl show nginx | grep -i nofile
LimitNOFILE=200000
DefaultLimitNOFILE=1024

# Kernel socket settings
sysctl net.core.somaxconn
sysctl net.ipv4.ip_local_port_range
net.core.somaxconn = 65535
net.ipv4.ip_local_port_range = 32768 60999
```

Why Connections Are Exhausted

- Each client connection consumes a worker connection.
- Upstream connections to backends also consume worker connections.
- Long-lived keepalive connections increase usage.
- file descriptor limits (ulimit / systemd) may be too low.
- TIME_WAIT sockets can temporarily consume ports.

Monitor Active Connections

```
# Nginx connection summary
sudo nginx -s status    # if stub_status enabled

# Using ss to check connections
sudo ss -s
sudo ss -nt state established '( sport = :80 or sport = :443 )'

# Check Nginx error log for connection limits
tail -n 50 /var/log/nginx/error.log
```

```
[error] 12345#0: *67890 accept() failed (24: Too many open files)
while accepting new connection
```

Configuration 13 — Control Request and Connection Concurrency

Scenario

A small number of clients overload an expensive API endpoint.

Key Points

1. Compare limit_req and limit_conn.
2. Configure appropriate request zones, burst behavior, and rejection responses.
3. Understand the effect of reverse proxies on client identification.
4. Avoid rejecting legitimate traffic behind shared IP addresses.
5. Validate limits with controlled load tests.

Nginx Rate and Connection Limiting Examples

```
http {
   # Define zones
   limit_req_zone  $binary_remote_addr zone=api_req:10m rate=10r/s;
   limit_conn_zone $binary_remote_addr zone=api_conn:10m;

   server {
      location /api/ {
         # Limit requests per second
         limit_req  zone=api_req  burst=20  nodelay;

         # Limit concurrent connections
         limit_conn  api_conn  10;
         proxy_pass http://api_backend;
      }
   }
```

limit_req vs limit_conn

limit_req (Request Rate)	limit_conn (Connection Count)
Limits requests per second from a client.	Limits the number of concurrent connections from a client.
Configured with rate and burst.	Helps prevent resource exhaustion.
Useful for API endpoints.	Useful for long-lived requests.
Returns 503 or 429 when exceeded.	Returns 503 when exceeded.

Handling Clients Behind Proxies

- Use real client IP (set_real_ip_from, real_ip_header).
- Consider using $binary_remote_addr with trusted proxies.
- Adjust limits for NAT, CDN, or load balancers.
- Avoid overly strict limits that block many legitimate users.

Test with Load Tools

```
# Test request rate limit
sudo wrk -t4 -c50 -d30s \
   http://example.com/api/

# Test connection limit
sudo hey -c 50 -n 1000 \
   http://example.com/api/
```

Expected Behavior

- Requests within limit succeed.
- Excess requests receive 429/503.
- Connections above limit are rejected.
- Legitimate users can still access services.

Senior Insight

Raising limits can postpone an outage without fixing the underlying saturation point.

VERIQTA

The TLS Configuration That Worked Until Certificate Rotation

Configuration 14 — Configure Production TLS Without Fragile Certificate Handling

Scenario

HTTPS fails after a certificate renewal or deployment.

Key Points

1. Configure ssl_certificate, ssl_certificate_key, and TLS protocol settings.
2. Examine certificate chains, private-key access, and SNI behavior.
3. Validate certificate deployment before reloading Nginx.
4. Investigate expired certificates and incorrect certificate selection.
5. Verify with openssl s_client and external HTTPS requests.

Nginx TLS Configuration Example

```
server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_certificate /etc/nginx/ssl/example.com/fullchain.pem;
    ssl_certificate_key /etc/nginx/ssl/example.com/privkey.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:
                 ECDHE-RSA-AES256-GCM-SHA384';
    ssl_prefer_server_ciphers off;
    ssl_session_timeout 1d;
    ssl_session_cache shared:SSL:10m;
    ssl_stapling on;
    ssl_stapling_verify on;
    resolver 1.1.1.1 8.8.8.8 valid=300s;
}
```

Verify Certificate with OpenSSL

```
openssl s_client -connect example.com:443 \\
  -servername example.com
CONNECTED(00000003)
depth=2 O = Let's Encrypt
depth=1 O = R3
depth=0 CN = example.com
---
Verify return code: 0 (ok)
```

Check Certificate Expiry

```
openssl x509 -in fullchain.pem -noout -text \
| grep -E 'Subject:|Issuer:|Not Before|Not After'
Subject: CN = example.com
Issuer:  CN = R3, O = Let's Encrypt
Not Before: Oct  1 00:00:00 2026 GMT
Not After : Dec 30 23:59:59 2026 GMT
```

Common Causes After Rotation

- ✗ Using only the leaf certificate (missing chain).
- ✗ Incorrect file paths or permissions.
- ✗ Private key does not match certificate.
- ✗ Old certificate still being loaded.
- ✗ SNI serving the wrong certificate.
- ✗ Certificate expired or not yet valid.

Validate Before Reloading Nginx

- ✓ Check certificate and key files exist and are readable.
- ✓ Verify certificate chain is complete.
- ✓ Test with: nginx -t
- ✓ Verify with: openssl s_client and external HTTPS request.
- ✓ Check the correct certificate is served for each domain (SNI).

Test with cURL (External)

```
curl -I https://example.com
HTTP/2 200
server: nginx
content-type: text/html
strict-transport-seurity:
max-age=63072000; includeSubDomains
```

Configuration 15 — Secure Backend TLS Connections

Scenario

Nginx connects to an HTTPS upstream, but backend certificate identity is not being verified.

Key Points

1. Configure proxy_ssl_verify on.
2. Set proxy_ssl_trusted_certificate and proxy_ssl_server_name on where appropriate.
3. Examine proxy_ssl_name when the upstream address differs from its certificate identity.
4. Distinguish encrypted transport from authenticated transport.
5. Verify backend certificate rejection and successful trusted connections.

Nginx Backend TLS Configuration

```
upstream backend_https {
    server api.internal.local:443;
}
server {
    listen 80;
    server_name example.com;

    location / {
        proxy_pass https://backend_https;
        proxy_ssl_verify on;
        proxy_ssl_trusted_certificate \
            /etc/nginx/ssl/internal-ca.pem;
        proxy_ssl_server_name on;
        proxy_ssl_name api.internal.local; # use cert CN/SAN
        proxy_set_header Host api.internal.local;
} }
```

Test Backend Certificate Verification

```
# Test connection and verify certificate
openssl s_client -connect api.internal.local:443 \
-servername api.internal.local -CAfile /etc/nginx/ssl/internal-ca.pem
```

Key Directives Explained

proxy_ssl_verify on	Verify the upstream server's certificate.
proxy_ssl_trusted_certificate	CA certificate used to verify the upstream certificate.
proxy_ssl_server_name on	Send SNI with the upstream hostname.
proxy_ssl_name	Specify the hostname to match the certificate (if different).

If Verification Fails

```
verify error:num=20:unable to get local issuer certificate
verify return code: 20 (unable to get local issuer certificate)
```

- ✗ Missing or incorrect CA certificate.
- ✗ Upstream certificate does not match hostname.
- ✗ SNI not set to the correct hostname.
- ✗ Self-signed certificate not trusted.

Senior Insight — HTTPS to an upstream does not automatically mean Nginx verifies that upstream's certificate.

VERIQTA

Security Headers That Broke Production Applications

Configuration 16 — Deploy Security Headers Without Breaking Application Behavior

Scenario

A security hardening change unexpectedly breaks browser functionality.

Key Points

1. Configure HSTS, content-type protections, and appropriate browser security headers.
2. Examine Content Security Policy compatibility with scripts and external resources.
3. Understand add_header inheritance and the always parameter.
4. Roll out restrictive policies with observation and testing.
5. Verify headers across successful and error responses.

Nginx Security Headers Example

```
server {
  listen 443 ssl http2;
  server_name example.com;

  # HSTS (1 year, include subdomains)
  add_header Strict-Transport-Security
    "max-age=31536000; includeSubDomains" always;

  # Prevent MIME type sniffing
  add_header X-Content-Type-Options "nosniff" always;
  # Clickjacking protection
  add_header X-Frame-Options  "SAMEORIGIN" always;

  # Referrer policy
  add_header Referrer-Policy "strict-origin-when-cross-origin"
   always;

  # Content Security Policy (adjust for your application)
  add_header Content-Security-Policy
    "default-src 'self';
     script-src 'self' https://cdn.example.com;
     style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;"
     always;
}
```

Important Security Headers

Strict-Transport-Security (HSTS)	Forces HTTPS connections (max-age, includeSubDomains).
X-Content-Type-Options	Prevents MIME type sniffing (nosniff).
X-Frame-Options	Protects against clickjacking (SAMEORIGIN or DENY).
Content-Security-Policy (CSP)	Controls which resources can be loaded.
Referrer-Policy	Controls referrer information sent to other sites.

Common Issues

- ✗ CSP blocking required scripts or external resources.
- ✗ HSTS causing issues on non-HTTPS subdomains.
- ✗ add_header not applied to error responses.
- ✗ Overly restrictive directives breaking application functionality.

Verify Headers

- ✓ Check headers in successful and error responses.
- ✓ Use browser developer tools (Network tab).
- ✓ Test with: curl -I https://example.com
- ✓ Validate CSP using browser console for violations.

Configuration 17 — Restrict Sensitive Endpoints Without Trusting Client-Controlled Headers

Scenario

An administrative endpoint is accidentally accessible through a proxy.

Key Points

1. Configure network-based access restrictions where appropriate.
2. Evaluate application authentication, mTLS, and identity-aware access.
3. Avoid relying on spoofable headers for authorization.
4. Understand access control behavior behind trusted proxies.
5. Test unauthorized access paths.

Nginx Access Restriction Example

```
# Restrict admin endpoint by trusted IP
location /admin/ {
    allow 10.0.0.0/8;        # internal network
    allow 192.168.0.0/16;    # internal network
    deny all;
    proxy_pass http://admin_backend;
}
# mTLS for sensitive endpoint
location /internal/ {
    ssl_verify_client on;
    ssl_client_certificate /etc/nginx/ssl/ca.pem;
    proxy_pass http://internal_service;
}

# Do NOT rely on client-controlled headers
# if ($http_x_forwarded_for = "10.0.0.1") { ... }  # Insecure
# Use real IP from trusted proxy instead (see configuration 04)
```

Access Control Options

IP Allow/Deny	Restrict based on trusted network ranges.
Application Auth	Use strong authentication (e.g., OAuth, SSO).
mTLS	Require client certificates for sensitive endpoints.
Identity-Aware Proxy	Use external identity providers (e.g., AWS ALB, Cloudflare Access).
Rate Limiting	Combine with limit_req / limit_conn for additional protection.

Avoid These Mistakes

- ✗ Relying on X-Forwarded-For for authorization.
- ✗ Trusting client-controlled headers.
- ✗ Exposing internal endpoints without authentication.
- ✗ Allowing unrestricted access through a reverse proxy.

Test Unauthorized Access

```
# Test from untrusted IP
curl -I https://example.com/admin/
# Test with spoofed header (should still be denied)
curl -I -H "X-Forwarded-For: 10.0.0.1" \
    https://example.com/admin/
```

Check Real Client IP

```
# Verify real client IP in logs
log_format main '$remote_addr - $remote_user
    [$time_local] "$request" $status';
# Test with trusted proxy configuration
# (see configuration 04)
```

Monitor and Verify

- ✓ Verify access logs for blocked requests.
- ✓ Test from different networks and IP addresses.
- ✓ Confirm that spoofed headers are ignored.
- ✓ Validate that legitimate access still works.

Senior Insight — Security controls must be validated against real request paths, not merely checked for their presence in configuration files.

VERIQTA

The Cache That Served One User's Data to Another

Configuration 18

Configure Proxy Caching Without Leaking Private Responses

Scenario

A cached API response exposes personalized information to another user.

Key Points

1. Configure proxy_cache_path, proxy_cache, and cache eligibility.
2. Examine cache keys, cookies, authorization, and response headers.
3. Understand the dangers of caching personalized responses.
4. Configure bypass and no-cache conditions where required.
5. Verify cache behavior using controlled users and response headers.

Nginx Proxy Cache Configuration Example

```
# Define cache storage
proxy_cache_path /var/cache/nginx/api_cache
    levels=1:2
    keys_zone=api_cache:100m
    max_size=10g
    inactive=60m
    use_temp_path=off;

server {
    listen 80;
    server_name api.example.com;

    location /api/ {
        proxy_cache api_cache;
        proxy_cache_key "$scheme$proxy_host$uri?$arg_stri
ng";
        proxy_cache_valid 200 10m;
        proxy_cache_valid 404 1m;
        proxy_cache_bypass $cookie_session $http_authoriza
tion;
        proxy_no_cache $cookie_session $http_authorization;
        add_header X-Cache-Status $upstream_cache_status
always;
        proxy_pass http://app_backend;
    }
}
```

Cache Eligibility and Key Considerations

proxy_cache_path	Defines cache storage, size, and key zone.
proxy_cache	Enables caching for a location.
proxy_cache_key	Determines what makes a response unique.
proxy_cache_valid	Sets how long responses are cached.
proxy_cache_bypass	Skips cache for specific conditions (e.g. cookies, authorization).
proxy_no_cache	Prevents storing response in cache.

Common Mistakes

- ✗ Caching responses with user-specific data.
- ✗ Using a cache key that does not include important parameters (e.g. query string, headers).
- ✗ Ignoring Authorization headers and cookies.
- ✗ Caching error responses or redirects.

Verify Cache Behavior

- ✓ Use multiple test users with different data.
- ✓ Check X-Cache-Status response header (HIT, MISS, BYPASS).
- ✓ Verify sensitive endpoints are not cached.
- ✓ Test with and without cookies or authorization headers.

Configuration 19

Prevent Cache Stampedes During Backend Recovery

Scenario

An expired cache entry causes thousands of simultaneous requests to hit a recovering application.

Key Points

1. Configure proxy_cache_lock for suitable workloads.
2. Examine proxy_cache_use_stale and proxy_cache_background_update.
3. Understand stale-response risks and cache-refresh behavior.
4. Evaluate data freshness against backend protection.
5. Measure cache hit ratios and origin request volume.

Cache Stampede Protection Configuration

```
proxy_cache_path /var/cache/nginx/api_cache
    levels=1:2
    keys_zone=api_cache:100m
    max_size=10g
    inactive=60m;

server {
    listen 80;
    server_name api.example.com;

    location /api/ {
        proxy_cache api_cache;
        proxy_cache_lock on;
        proxy_cache_lock_timeout 10s;
        proxy_cache_use_stale error timeout
        updating http_500 http_502 http_503 http_504;
        proxy_cache_background_update on;
        proxy_cache_valid 200 10m;
        proxy_pass http://app_backend;
        add_header X-Cache-Status $upstream_cache_status
    } always;
```

Key Directives for Stampede Protection

proxy_cache_lock	Allows only one request to refresh an expired cache entry while others wait.
proxy_cache_lock_timeout	Time to wait for the cache lock before serving a stale response.
proxy_cache_use_stale	Serves stale content when the backend is unavailable or slow.
proxy_cache_background_update	Refreshes the cache in the background after serving stale content.

Benefits and Trade-offs

- ✓ Prevents sudden spikes to the backend.
- ✓ Keeps serving content during backend recovery.
- ✓ Reduces latency for most users.
- ✓ Must evaluate data freshness requirements.
- ✓ Stale content may be served temporarily.

Measure Cache Performance

```
# Check cache status in access logs
log_format main '$remote_addr - $upstream_cache_status';

# Analyze cache hit ratio
sudo awk '{print $10}' /var/log/nginx/access.log |
  sort | uniq -c
```

Test with Load

```
# Simulate many simultaneous requests
for an expired entry
for i in {1..100}; do
  curl -s http://api.example.com/api/data
  >/dev/null &
done; wait
```

Expected Behavior

- ✓ Only one request fetches fresh content from the backend.
- ✓ Other requests receive cached or stale content.
- ✓ Backend is protected from a sudden spike.
- ✓ Cache is updated in the background.

Senior Insight

A cache can become a failure amplifier if expiration causes synchronized backend demand.

VERIQTA

Nginx Rewrites That Silently Changed Application Behavior

Configuration 20 — Fix URI Rewriting and Proxy Path Semantics

Scenario

Requests reach the backend with unexpected or duplicated path segments.

Key Points

1. Understand proxy_pass behavior with and without a URI component.
2. Examine prefix-location matching and URI replacement.
3. Compare rewrite, return, and direct proxy routing.
4. Avoid accidental redirect loops and path confusion.
5. Verify upstream request paths using application logs.

proxy_pass Rehavior Examples

```
# Without URI component (passes original URI)
location /app/ {
    proxy_pass http://backend;
}

# With URI component (replaces matching prefix)
location /app/ {
    proxy_pass http://backend/;
}

# Rewrite example (change URI)
location /old/ {
    rewrite ^/old/(.*)$ /new/$1 break;
    proxy_pass http://backend;
}

# Return example (redirect)
location /app {
    return 301 /app/;
}
```

Key Differences

proxy_pass http://backend	Passes the original request URI (/app/users).
proxy_pass http://backend/	Replaces the matching prefix (/app/ → /).
rewrite	Modifies the URI using rules.
return	Sends an HTTP redirect or response code.
Direct proxy routing	Forwards without modification.

Common Mistakes

- Duplicated path segments (e.g. /app/app/).
- Missing or extra trailing slashes.
- Unexpected redirects and rewrite loops.
- Incorrect assumption about proxy_pass behavior.
- Not verifying the actual URI received by the backend.

Verify Correct Upstream Paths

- Check application access logs for the received URI.
- Use curl to test actual upstream paths.
- Test different path variations (with and without slashes).
- Verify there are no redirect loops.

Configuration 21 — Serve Static Assets Without Exposing Sensitive Files

Scenario

An Nginx static-file configuration unintentionally exposes files outside the intended public directory.

Key Points

1. Compare root and alias.
2. Examine trailing-slash behavior and location matching.
3. Restrict sensitive file paths and directories.
4. Validate filesystem permissions and Nginx path resolution.
5. Test traversal attempts and unexpected file exposure.

root vs alias Examples

```
# Using root (appends the full request URI)
location /static/ {
    root  /var/www/app;
    # Request: /static/css/style.css
    # File: /var/www/app/static/css/style.css
}
# Using alias (replaces the matching prefix)
location /assets/ {
    alias /var/www/app/assets/;
    # Request: /assets/img/logo.png
    # File: /var/www/app/assets/img/logo.png
}
# Correct trailing slash with alias
location /assets/ {
    alias /var/www/app/assets/; # include trailing slash
}
# Incorrect alias (missing trailing slash)
location /assets/ {
    alias  /var/www/app/assets;  # can cause path issues
}
```

Important Considerations

root	Appends the full request URI to the specified directory.
alias	Replaces the matching location prefix with the specified directory.
Trailing slash	alias should usually end with /.
Location matching	More specific locations take precedence.
File permissions	Ensure sensitive files are not readable by the Nginx user.

Security Measures

- Block access to sensitive files (e.g. .env, .git, config).
- Use location rules to deny access to hidden files.
- Validate directory permissions and ownership.
- Restrict directory listing (autoindex off).
- Prevent path traversal attempts.

Test for File Exposure

```
# Try to access sensitive files
curl -I https://example.com/.env
curl -I https://example.com/../etc/passwd
# Check response code (should be 403 or 404)
```

Expected Behavior

- Public assets are served correctly.
- Sensitive files return 403 or 404.
- No directory traversal is possible.
- Only intended files are accessible.

Additional Best Practices

- Use try_files for controlled file serving.
- Disable unnecessary HTTP methods.
- Keep sensitive files outside the web root.
- Regularly test with security scanning tools.

Senior Insight

URI routing and filesystem path construction are security-sensitive behaviors, not cosmetic configuration details.

The Logs Say 200, but Customers Report Failure

Configuration 22 — Build Nginx Access Logs for Production Root-Cause Analysis

Scenario

The access log reports successful HTTP responses while users experience high latency and failed workflows.

Key Points

1. Configure a structured log_format.
2. Capture request time, upstream response time, upstream address, and upstream status.
3. Distinguish Nginx response codes from upstream outcomes.
4. Correlate reverse proxy evidence with application logs and traces.
5. Avoid logging credentials, tokens, or sensitive query parameters.

Structured Access Log Example

```
# Define a structured log format
log_format main_ext
 {
     "time":"$time_iso8601",
     "remote_addr":"$remote_addr",
     "request":"$request",
     "status":$status,
     "body_bytes_sent":$body_bytes_sent,
     "request_time":$request_time,
     "upstream_addr":"$upstream_addr",
     "upstream_status":"$upstream_status",
     "upstream_response_time":"$upstream_response_time",
     "http_referer":"$http_referer",
     "http_user_agent":"$http_user_agent"
 }

 access_log  /var/log/nginx/access.log  main_ext;
```

Sample Log Entry (Formatted)

```
{"time":"2026-10-27T12:30:45+00:00","remote_addr":"203.0.113.10",
"request":"GET /api/orders HTTP/1.1","status":200,
"body_bytes_sent":532,"request_time":0.412,
"upstream_addr":"10.0.1.10:8080","upstream_status":"500",
"upstream_response_time":"0.401","http_referer":"-",
"http_user_agent":"curl/8.4.0"}
```

Important Log Fields

request_time	Total time taken to process the request in Nginx.
upstream_response_time	Time taken by the upstream application to respond.
upstream_addr	The upstream server address that handled the request.
upstream_status	Response status from the upstream server.
status	Nginx response code to the client (may be 200 even if upstream failed).

Common Mistakes

- Relying only on Nginx status code (e.g., 200).
- Not logging upstream response time or status.
- Logging sensitive information (tokens, passwords).
- Using an unstructured log format.
- Not correlating with application logs and traces.

Verify and Analyze

- Check logs for request_time and upstream_response_time.
- Identify mismatched Nginx and upstream status.
- Correlate timestamps (e.g. application logs).
- Ensure no sensitive data is logged.

Configuration 23 — Expose Nginx Health and Capacity Signals

Scenario

CPU dashboards appear normal while connections and request latency deteriorate.

Key Points

1. Configure stub_status where supported.
2. Restrict status endpoint access.
3. Understand active, reading, writing, and waiting connection counters.
4. Combine Nginx metrics with operating-system and upstream telemetry.
5. Design alerts based on symptoms rather than process availability alone.

Nginx stub_status Configuration

```
# Enable stub_status on a restricted location
server {
    listen 80;
    server_name example.com;

    location /nginx_status {
        stub_status;
        allow 10.0.0.0/8;     # internal network
        allow 192.168.00/16; # internal network
        deny all;
    }
}
```

Example Output

```
Active connections: 291
server accepts handled requests
 16384 16384 310704
Reading: 12 Writing: 8 Waiting: 271
```

Connection State Meaning

Active connections	Total current connections (open and processing).
Reading	Connections reading request from client.
Writing	Connections writing response to client.
Waiting	Keep-alive connections, idle and ready for new requests.

What to Monitor

- Active connections trending upward.
- High number of reading or writing connections.
- Increase in waiting connections (possible client issues).
- Correlate with CPU, memory, file descriptors, and upstream metrics.
- Set alerts for unusual patterns (latency, error rates, connection count).

Senior Insight

Useful observability reveals where time and resources are consumed across the request path.

The 504 Incident That Was Actually a Database Problem

Configuration 24

Investigate Upstream Timeouts Without Random Configuration Changes

Scenario

Nginx returns intermittent 504 Gateway Timeout responses during peak traffic.

Key Points

1. Distinguish connection failures, upstream read timeouts, and backend processing delays.
2. Correlate Nginx timing variables with database connection saturation.
3. Examine application worker pools and dependency latency.
4. Identify whether the bottleneck is Nginx, the application, or a downstream service.
5. Mitigate safely before considering timeout changes.

Understanding 504 Errors

504 Nginx returns 504 Gateway Timeout when it does not receive a timely response from an upstream server.

Common Causes

- Upstream server not reachable (connection failure).
- Upstream server slow to respond (upstream read timeout).
- Application worker pool exhausted.
- Database connection saturation or slow queries.
- High latency in downstream services (e.g. cache, message queue, external API).

Nginx Timing Variables

$request_time	Total time to process the client request.
$upstream_connect_time	Time to establish connection to upstream.
$upstream_header_time	Time to receive response headers from upstream.
$upstream_response_time	Time to receive full response from upstream.
$upstream_addr	The upstream server address that handled the request.
$upstream_status	Response status from the upstream server.

Production Investigation Commands

1. Test Nginx Configuration

```
$ sudo nginx -t
nginx: the configuration file
/etc/nginx/nginx.conf
syntax is ok
nginx: configuration file
/etc/nginx/nginx.conf
test is successful
```

What It Establishes

- Verifies configuration syntax.
- Confirms recent changes are valid.
- Does not check upstream health or runtime behavior.

2. Check Recent Error Logs

```
$ sudo tail -n 100
 /var/log/nginx/error.log
2026/10/27 14:32:11 [error]
12345#12345: *987 upstream
timed out (110: Connection
timed out) while reading
response header from upstream,
client: 203.0.113.10,
server: example.com,
request: "GET /api/data
HTTP/1.1", upstream:
"http://10.0.1.20:8080/api/data".
```

What It Establishes

- Reveals connection failures, timeouts, and upstream errors.
- Shows the exact error type and upstream server involved.
- Combines with access logs for full request context.

3. Check Connection Summary

```
$ sudo ss -s
Total: 12567 (kernel 13000)
TCP:  8900 (estab 8300,
closed 520, orphaned 12,
timewait 2100)
Transport Total     IP      IPv6
*         12567     -       -
RAW       12        8       4
UDP       320       290     30
TCP       12235     11890   345
```

What It Establishes

- Shows current socket and connection counts.
- Helps identify high numbers of established or time-wait connections.
- Indicates whether Nginx is under connection pressure.
- Does not show application or database layer bottlenecks.

Additional Evidence Required

- Application logs (worker errors, request processing time).
- Database metrics (active connections, slow queries, query latency).
- Upstream service metrics (response time, error rate).
- System metrics (CPU, memory, disk I/O, network latency).
- Distributed tracing (e.g. OpenTelemetry) for full request path.

Investigation Steps

1. Check Nginx error and access logs for the exact 504 entries.
2. Analyze Nginx timing variables to see where time is spent.
3. Correlate with application logs and database metrics.
4. Determine whether the issue is at Nginx, the application, or a downstream service.
5. Mitigate the root cause before increasing timeouts.

Senior Insight

Nginx often reports the symptoms of failures originating several dependencies away.

The Nginx Reload That Broke Production Traffic

Configuration 25

Deploy and Reload Nginx Configuration Safely

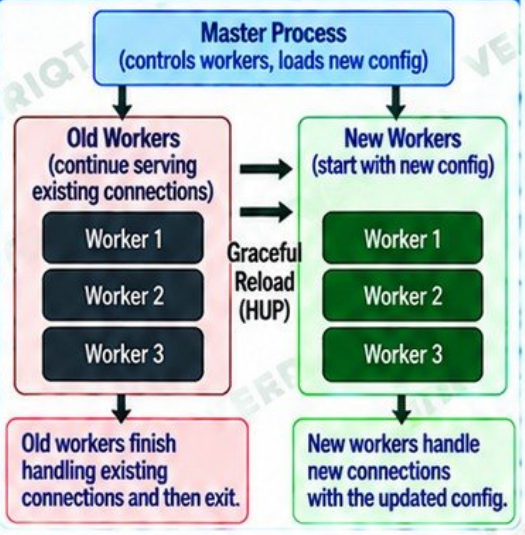
Scenario

A configuration deployment causes request failures even though the configuration passes syntax validation.

Key Points

1. Understand Nginx master and worker process behavior during reload.
2. Distinguish syntax validation from functional correctness.
3. Test configuration using nginx -t.
4. Examine long-lived connections and worker shutdown behavior.
5. Deploy changes incrementally where the architecture permits.
6. Verify response behavior, error rates, and connection health after reload.

Nginx Reload Process



Important Commands

sudo nginx -t	Test configuration syntax.
sudo nginx -s reload	Reload Nginx with new configuration.
sudo nginx -s stop	Stop Nginx gracefully (after connections finish).
sudo nginx -s quit	Graceful shutdown (used during maintenance).
sudo systemctl status nginx	Check current Nginx status and processes.
ps -ef | grep nginx	View master and worker processes.
sudo ss -tnp | grep nginx	Check active connections.

Example: Safe Configuration Test and Reload

```
# 1. Test configuration syntax
$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

# 2. Reload Nginx
$ sudo nginx -s reload

# 3. Verify processes
$ ps -ef | grep nginx
nginx    1234     1  0 10:00 ?     00:00:00 nginx: master process /usr/sbin/nginx
nginx    1235  1234  0 10:00 ?     00:00:00 nginx: worker process
nginx    1236  1234  0 10:00 ?     00:00:00 nginx: worker process
```

What to Understand

1. The master process reads the new configuration and starts new workers.
2. Old workers continue handling existing connections.
3. Workers shut down gracefully after completing active requests.
4. Long-lived connections (e.g. WebSocket, HTTP/2, keep-alive) may persist on old workers.
5. Reload does not drop active connections but may expose issues in the new config.

Common Causes of Failures After Reload

- Configuration is syntactically correct but functionally wrong (e.g. wrong upstream).
- Upstream servers are unreachable.
- Incorrect SSL certificate or key paths.
- Missing or incorrect include files.
- Unintended redirect or rewrite rules.
- Access control rules blocking legitimate traffic.
- Changes to headers, timeouts, or buffering that break application behavior.

Post-Reload Verification

- Check access and error logs for new errors.
- Monitor HTTP status codes and error rates.
- Verify upstream connectivity.
- Check active connections and worker processes.
- Test key application endpoints.
- Confirm long-lived connections are stable.
- Monitor metrics (latency, upstream response time, connection count).

Senior Insight

A successful syntax check proves that Nginx accepts the configuration, not that the configuration preserves application behavior.

One Shared Nginx Configuration Became a Platform-Wide Failure

Configuration 26 — Design Safe Multi-Service Nginx Configuration Management

Scenario

A shared Nginx configuration change breaks several unrelated applications.

Key Points

1. Structure configuration files with controlled includes.
2. Establish service ownership and change boundaries.
3. Examine global directives that unintentionally affect multiple virtual hosts.
4. Use automated configuration testing and deployment approvals.
5. Design rollback and independent failure domains.
6. Compare centralized gateway management with per-service proxy ownership.

Recommended Configuration Structure

```
/etc/nginx/
├─ nginx.conf          main configuration
├─ conf.d/             # global configs (careful)
│  ├─ global/          # shared but controlled
│  │  ├─ upstreams.conf
│  │  ├─ security.conf
│  │  └─ logging.conf
├─ sites-available/    # per-service configs
│  ├─ app1.conf
│  ├─ app2.conf
│  └─ app3.conf
└─ sites-enabled/      # enabled services (symlinks)
   ├─ app1.conf -> ../sites-available/app1.conf
   ├─ app2.conf -> ../sites-available/app2.conf
   └─ app3.conf -> ../sites-available/app3.conf
```

Example Per-Service Configuration

```
# /etc/nginx/sites-available/app1.conf
server {
    listen 80;
    server_name app1.example.com;

    location / {
      proxy_pass http://app1_backend;
      include /etc/nginx/conf.d/global/security.conf;
    }
}

# /etc/nginx/sites-available/app2.conf
server {
    listen 80;
    server_name app2.example.com;

    location / {
      proxy_pass http://app2_backend;
      include /etc/nginx/conf.d/global/security.conf;
    }
}
```

Global Directives That Can Cause Platform-Wide Impact

Directive	Potential Impact
worker_processes	Affects all services.
events (worker_connections)	Limits total connection capacity.
http { ... } settings	Affects all virtual hosts unless overridden.
include files	Changes propagate to every server that includes them.
map, upstream, log_format	Can break multiple services if modified incorrectly.
SSL protocols and ciphers	May affect all HTTPS services.
Security headers	Can change behavior for all applications.

Configuration Testing and Deployment

1. Use nginx -t to validate syntax.
2. Test configuration in a staging environment.
3. Use automated tests in CI/CD pipelines.
4. Require peer review and approval for shared changes.
5. Deploy incrementally where possible.
6. Verify application health after deployment.
7. Have a quick rollback plan.

Centralized Gateway vs Per-Service Proxy

Centralized Gateway (Shared Nginx)

- ✓ Single entry point and simplified management.
- ✓ Consistent security and logging.
- ✓ Easier to enforce global policies.
- ✗ Higher risk of correlated failures.
- ✗ Changes can affect multiple services.
- ✗ Requires strong change control and testing.

Per-Service Proxy (Dedicated Nginx)

- ✓ Independent failure domains.
- ✓ Service teams own their configuration.
- ✓ Changes are isolated to a single service.
- ✗ More infrastructure to manage.
- ✗ Inconsistent configurations if not well governed.
- ✗ Higher operational overhead.

Example Validation Commands

```
# Test configuration syntax
sudo nginx -t

# Check what configuration files are loaded
sudo nginx -T | less

# Check Nginx processes
ps -ef | grep nginx

# Check listening ports
sudo ss -lntp | grep nginx

# Check recent errors after reload
sudo tail -n 100 /var/log/nginx/error.log
```

Senior Insight

Configuration reuse reduces duplication but can increase correlated failure risk.

WebSockets Disconnect While Ordinary HTTP Requests Work

Configuration 27

Configure WebSocket Proxying Correctly

Scenario

Real-time application connections disconnect unexpectedly behind Nginx, while ordinary HTTP requests work normally.

Key Points

1. Configure HTTP upgrade handling and connection headers.
2. Examine proxy_http_version, Upgrade, and Connection directives.
3. Understand idle timeouts and long-lived connection behavior.
4. Investigate load balancer and application timeout interactions.
5. Verify successful protocol upgrades and connection stability.

How WebSocket Proxying Works



Correct Nginx Configuration Example

```
map $http_upgrade $connection_upgrade {
    default  upgrade;
    ''       close;
}

upstream websocket_backend {
    server 10.0.0.20:8080;
}

server {
    listen 80;
    server_name example.com;

    location /ws/ {
        proxy_pass http://websocket_backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
        proxy_buffering off;
    }
```

Important Directives

proxy_http_version 1.1	Required for WebSocket upgrade.
proxy_set_header Upgrade	Passes the Upgrade header to the backend.
proxy_set_header Connection	Set to upgrade for WebSocket connections.
proxy_read_timeout	Controls how long Nginx waits for data from the backend.
proxy_send_timeout	Controls how long Nginx waits to send data to the backend.
proxy_buffering off	Disables response buffering for real-time communication.

Common Issues and Fixes

- Missing Upgrade or Connection headers.
- Using HTTP/1.0 instead of 1.1.
- Idle timeouts closing connections.
- Load balancer timeouts (e.g. AWS ALB, ELB).
- proxy_buffering enabled.
- Application server closing idle connections.
- Firewall or security groups terminating long-lived connections.

Test WebSocket Connection

```
# Test using wscat (Node.js)
wscat -c ws://example.com/ws/

# Test using curl (check upgrade response)
curl -i -N -H "Connection: Upgrade" \
  -H "Upgrade: websocket" \
  http://example.com/ws/

# Expected response (101 Switching Protocols)
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
```

Verify in Nginx Logs

```
# Access log should show 101 status
203.0.113.10 - - [27/Oct/2026:14:10:05
+0000] "GET /ws/ HTTP/1.1" 101 -
"-" "Mozilla/5.0"
```

Check Active Connections

```
# Monitor active connections
sudo ss -tnp | grep nginx

# Watch Nginx connection counters
watch -n 2 'ss -s'
```

Additional Considerations

- Increase timeouts for long-lived connections.
- Ensure load balancer and firewall timeouts are higher than Nginx timeouts.
- Test with real client application (browser or WebSocket client).
- Monitor connection count and resource usage.
- Check application server logs for disconnect reasons.
- Consider sticky sessions if required by the application.

Senior Insight

WebSocket connections are long-lived and require proper upgrade handling, timeout configuration, and end-to-end alignment across proxies, load balancers, and application servers.

HTTP/2 Improved Client Performance but Did Not Fix the Backend

Configuration 28

Configure HTTP/2 and gRPC Routing Without Protocol Confusion

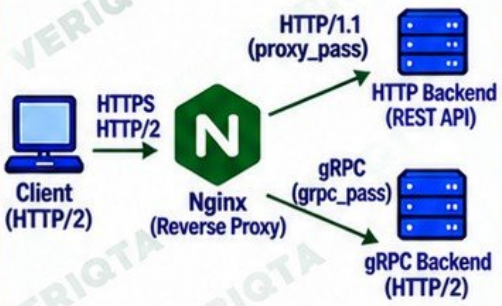
Scenario

HTTP/2 clients connect successfully, but gRPC requests fail through the proxy.

Key Points

1. Distinguish client-facing HTTP/2 from upstream protocol handling.
2. Compare proxy_pass with grpc_pass.
3. Examine gRPC status codes, streaming, and deadlines.
4. Configure TLS and upstream protocol behavior appropriately.
5. Investigate intermediary timeouts and unsupported protocol assumptions.
6. Verify gRPC behavior with appropriate diagnostic clients.

Architecture Overview



Important Differences

Feature	HTTP (proxy_pass)	gRPC (grpc_pass)
Protocol	HTTP/1.0 or 1.1	HTTP/2 (required)
Nginx directive	proxy_pass	grpc_pass
Use case	REST, web apps, standard HTTP	gRPC services
Connection handling	Request/response	Bidirectional streaming (HTTP/2)
Status codes	HTTP status codes	gRPC status codes (e.g. UNAVAILABLE)
Streaming	Limited (chunked)	Full streaming (client, server, bidirectional)

Nginx HTTP/2 Listener (Client-Facing)

```
server {
    listen 443 ssl http2;
    server_name api.example.com;

ssl_certificate /etc/ssl/certs/api.crt;
ssl_certificate_key /etc/ssl/private/api.key;
ssl_protocols TLSv1.2 TLSv1.3;

# route HTTP requests
location /api/ {
    proxy_pass http://http_backend;
}

# route gRPC requests
location /grpc/ {
    grpc_pass grpc://grpc_backend;
}
```

Upstream Configuration

```
upstream http_backend {
    server 10.0.0.10:8080;
    server 10.0.0.11:8080;
}
upstream grpc_backend {
    server 10.0.0.20:50051;
    server 10.0.0.21:50051;
}
```

gRPC Location Block Example

```
location /grpc/ {
    grpc_pass grpc://grpc_backend;
    grpc_read_timeout 3600s;
    grpc_send_timeout 3600s;
    grpc_set_header Host $host;
    grpc_set_header X-Real-IP $remote_addr;
    grpc_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
```

Test and Verify gRPC

```
# Test gRPC service using grpcurl
grpcurl -v -insecure \
   -d '{}' api.example.com:443 \
  service.package.Service/Method

# List available services
grpcurl -insecure api.example.com:443 list
```

Common gRPC Errors

- UNAVAILABLE – upstream not reachable or timeout.
- DEADLINE_EXCEEDED – request took longer than allowed.
- INTERNAL – application error.
- protocol error – HTTP/1.1 used instead of HTTP/2.
- invalid content-type – missing gRPC content type.

Investigation Steps

1. Verify client is using HTTP/2.
2. Confirm grpc_pass is used for gRPC.
3. Check Nginx error logs for gRPC errors.
4. Validate upstream service is reachable.
5. Test with grpcurl or a gRPC client.
6. Review timeouts, load balancer settings, and TLS configuration.

Senior Insight

Enabling HTTP/2 on the client-facing listener does not automatically make ordinary HTTP proxying equivalent to gRPC proxying.

The Nginx Performance Optimization That Increased Memory Pressure

Configuration 29

Tune Proxy Buffering Based on Measured Workload Behavior

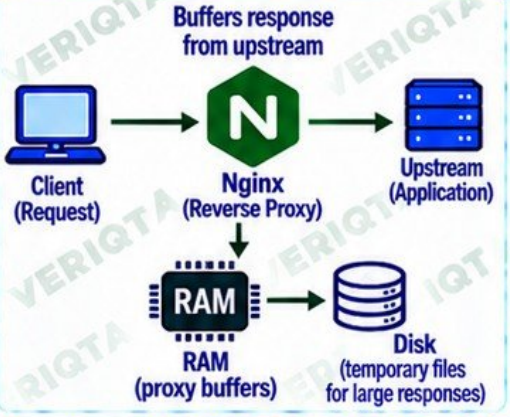
Scenario

Engineers increase proxy buffers to improve throughput, but memory consumption rises sharply under concurrency.

Key Points

1. Examine proxy_buffering, proxy_buffers, and proxy_buffer_size.
2. Understand response buffering and temporary-file behavior.
3. Compare buffering enabled versus disabled for streaming workloads.
4. Evaluate memory consumption under concurrent requests.
5. Measure throughput, latency, disk I/O, and resource saturation.
6. Avoid blindly copying performance-tuning snippets from the internet.

How Proxy Buffering Works



Key Directives

Directive	Example	Purpose
proxy_buffering	on | off	Enable or disable response buffering.
proxy_buffers	8 16k;	Number and size of buffers per request.
proxy_buffer_size	16k;	Buffer for the first part of the response (header + body).
proxy_busy_buffers_size	32k;	Size of busy buffers that cannot be reused.
proxy_temp_path	/var/lib/nginx/proxy_temp	Location for temporary files when response exceeds buffers.

Example: Buffering Enabled (Standard API Response)

```
location /api/ {
    proxy_pass http://app_backend;
    proxy_buffering on;
    proxy_buffers 8 16k;
    proxy_buffer_size 16k;
    proxy_busy_buffers_size 32k;
    proxy_temp_path
      /var/lib/nginx/proxy_temp;
}
```

Example: Buffering Disabled (Streaming / Real-Time)

```
location /stream/ {
    proxy_pass http://stream_backend;
    proxy_buffering off;
    proxy_request_buffering off;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
}
```

Buffering Behavior Comparison

Aspect	Buffering Enabled	Buffering Disabled
Response handling	Buffers response in memory (and disk if large).	Streams directly to client.
Memory usage	Higher (per concurrent request).	Lower.
Disk I/O	May use temporary files for large responses.	No temporary files.
Best for	Typical web APIs, static content.	Streaming (video, WebSockets, gRPC, SSE).
Risk	High memory usage under concurrency.	Higher upstream load, lower client buffering.

Monitor Resource Usage

```
# Check Nginx memory usage
ps -eo pid,ppid,cmd,%mem,%cpu | grep nginx
# Monitor memory in real time
top -p $(pgrep nginx)
# Check temporary files
ls -lh /var/lib/nginx/proxy_temp
# Monitor disk I/O
sudo iostat -xz 1
```

Common Problems

- ✗ Excessive memory usage under high concurrency.
- ✗ Disk usage increases due to temporary files.
- ✗ Increased latency from disk I/O.
- ✗ Buffer sizes copied from the internet without testing.
- ✗ Buffering enabled for streaming workloads.
- ✗ Resource saturation affecting other services.

Tuning Checklist

- ✓ Understand your workload and response sizes.
- ✓ Test different buffer sizes under realistic load.
- ✓ Monitor memory, disk I/O, and latency.
- ✓ Use streaming configuration where appropriate.
- ✓ Set dedicated proxy_temp_path with sufficient storage.
- ✓ Validate changes in a staging environment before production.

Senior Insight

Performance tuning must consider concurrency and aggregate resource consumption, not just single-request latency.

VERIQTA

The Nginx Deployment That Could Not Survive a Backend Failure

Configuration 30

Design Failure-Aware Nginx Upstream Routing

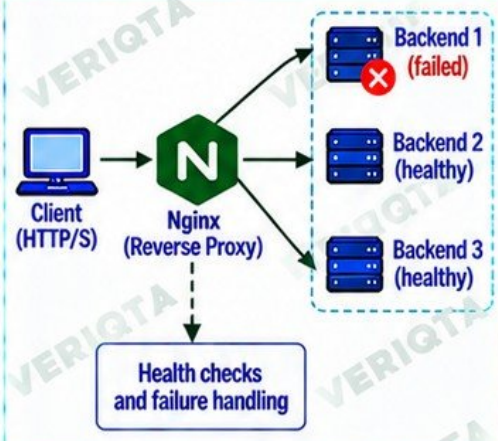
Scenario

A critical application runs behind Nginx with multiple backends, but one backend failure still causes widespread request errors.

Key Points

1. Combine upstream failure handling, connection timeouts, and bounded retries.
2. Examine upstream failure detection limitations.
3. Distinguish connection failures from application-level errors.
4. Decide when Nginx should retry, reject, or allow the application to handle failure.
5. Evaluate load balancing behavior during partial outages.
6. Test backend failure scenarios before production approval.

Architecture Overview



Key Directives

Directive	Example	Purpose
upstream	upstream app { ... }	Define multiple backends.
server	server 10.0.0.1:8080;	Backend server address.
proxy_connect_timeout	5s;	Time to establish connection.
proxy_read_timeout	30s;	Time to read response.
proxy_next_upstream	error timeout http_502 http_503 http_504;	Conditions for retry on another backend.
proxy_next_upstream_tries	3;	Maximum retry attempts.
proxy_next_upstream_timeout	10s;	Total time for retries.
ip_hash / least_conn	least_conn;	Load balancing method (example).

Example: Failure-Aware Upstream Configuration

```
upstream app_backend {
    least_conn;
    server 10.0.0.1:8080 max_fails=3 fail_timeout=30s;
    server 10.0.0.2:8080 max_fails=3 fail_timeout=30s;
    server 10.0.0.3:8080 max_fails=3 fail_timeout=30s;
}

server {
    listen 443 ssl http2;
    server_name app.example.com;

    location / {
        proxy_pass http://app_backend;
        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
        proxy_next_upstream error timeout http_502 http_503 http_504;
        proxy_next_upstream_tries 3;
        proxy_next_upstream_timeout 10s;
    }
}
```

Upstream Failure Conditions

Condition	Description
error	Network or internal error (502, 503, 504 from upstream).
timeout	Connection, read, or send timeout.
invalid_header	Invalid response from upstream.
http_500	Retry on 500 (optional).
http_502	Retry on 502.
http_503	Retry on 503.
http_504	Retry on 504.
non_idempotent	Prevent retry for non-idempotent requests (e.g. POST).

Load Balancing Methods

Method	Use Case
round_robin	Default, general purpose.
least_conn	Better for variable request times.
ip_hash	Keep same client on the same backend.
hash	Custom key-based routing.
backup	Standby server used only when others fail.

Test Backend Failure Scenario

```
# Simulate failure on backend 1
sudo systemctl stop app1

# Monitor Nginx error log
sudo tail -f /var/log/nginx/error.log

# Generate test traffic
for i in {1..10}; do
  curl -s -o /dev/null -w "%{http_code}\n"
  https://app.example.com;
done
```

Expected Behavior

- Nginx stops sending traffic to the failed backend after max_fails is reached.
- Requests are routed to healthy backends.
- Retries occur only for configured failure conditions.
- Client receives successful responses if healthy backends are available.
- Error rates are limited during partial outages.

Common Issues and Fixes

- All backends marked as failed
  → Check health check logic and timeouts.
- Retries on non-idempotent requests
  → Avoid retrying POST/PUT unless safe.
- Slow failover
  → Reduce timeouts and max_fails.
- High error rates during partial outage
  → Adjust load balancing method and retry settings.
- Application-level errors not retried
  → Decide if Nginx should retry or application should handle errors.

Senior Insight

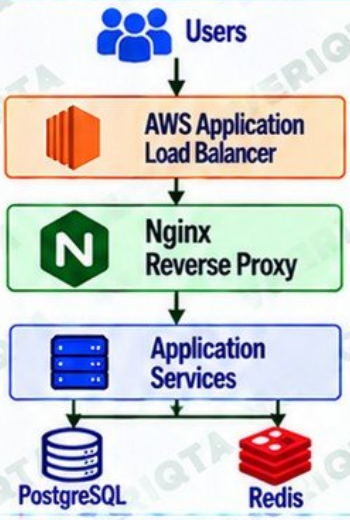
Reliability requires coordinated behavior across Nginx, applications, infrastructure, and dependencies. No single directive can guarantee it.

The Nginx Production Incident: Diagnose Before You Change Anything

Final Scenario

Real-World Incident Analysis and Resolution

Architecture Overview



Incident Timeline and Evidence

Time	Event
10:27 a.m.	Application deployments completed (15 minutes before incident).
10:42 a.m.	Customers begin reporting intermittent failures.
Nginx responses	Returns both 502 (Bad Gateway) and 504 (Gateway Timeout).
Response times	Some requests < 100 ms. Others > 30 seconds.
Nginx CPU	Moderate utilization.
Active connections	Increase sharply.
Database connections	Approach configured limit.

Reported Symptoms

- ✗ Intermittent request failures.
- ✗ Nginx 502 and 504 responses.
- ✗ Some requests complete quickly (< 100 ms).
- ✗ Some requests take > 30 seconds.
- ✓ Nginx CPU remains moderate.
- ✗ Active connections increase sharply.
- ✗ Database connections near limit.
- ✗ Recent application deployment (15 minutes before incident).
- ⚠ Engineers propose increasing Nginx timeouts.

Example Nginx Access Log (Relevant Fields)

```
203.0.113.10 - - [20/Oct/2026:10:42:12 +0000]
"GET /api/orders HTTP/1.1" 502 173 "-" "-"
0.031 0.030 - -

203.0.113.11 - - [20/Oct/2026:10:42:15 +0000]
"POST /api/payments HTTP/1.1" 504 560 "-" "-"
30.125 30.000 10.0.1.23:8080 504

203.0.113.12 - - [20/Oct/2026:10:42:18 +0000]
"GET /api/products HTTP/1.1" 200 932 "-" "-"
0.082 0.080 10.0.1.21:8080 200
```

Key Metrics During Incident

Request latency	< 100 ms to > 30 seconds
Nginx CPU utilization	Moderate (around 30–40%)
Active connections	Sharp increase (from ~2K to > 15K)
Application response time	Increased and unstable
Database connections	Near configured limit (e.g. 450/500)
Error rate (502/504)	Signficant increase

Possible Sources of the Errors

- **Nginx reverse proxy** (connection, timeout, upstream).
- **Application services** (worker saturation, slow processing).
- **PostgreSQL** (connection limit, slow queries).
- **Redis** (latency, connection issues).
- **AWS Application Load Balancer** (health checks, target availabiilty).
- **Recent deployment** (configuration or code changes).

Senior Engineering Investigation Steps

1. Define the customer impact and affected request paths.
2. Determine whether the errors originate from Nginx, upstream applications, or downstream dependencies.
3. Correlate Nginx request timings with application and database metrics.
4. Investigate the recent deployment without assuming it caused the incident.
5. Examine connection pooling, database saturation, and request queueing.
6. Evaluate whether increasing timeouts would worsen resource exhaustion.
7. Select the safest mitigation (rollback, traffic reduction, or dependency recovery).
8. Verify reduced errors, restored latency, and stable connection counts.
9. Identify the root cause and distinguish it from contributing factors.
10. Define prevention through observability, capacity controls, deployment safety, and tested recovery procedures.

Key Diagnostic Commands

```
# Check Nginx active connections
sudo nginx -s status
# View recent Nginx error log entries
sudo tail -n 100 /var/log/nginx/error.log
# Check Nginx access log for upstream timings
sudo tail -n 100 /var/log/nginx/access.log
# Check application service status
sudo systemctl status app.service
# Check database connections (PostgreSQL)
sudo -u postgres psql -c "select count(*) from pg_stat_activity;"
# Check Redis connected clients
redis-cli info clients
# Check system resource usage
top -p $(pgrep nginx),$(pgrep app),$(pgrep postgres)
```

Senior Insight

Reliability requires coordinated behavior across Nginx, applications, infrastructure, and dependencies. No single directive can guarantee it.